```
  if Rails.env.test?
    namespace :test do
      post("log_in_user", to: "setup#log_in_user")
      post("add_favorite", to: "setup#add_favorite")
    end
  end
end
```

The routing uses the Rails namespace feature to put the two routes inside a common namespace. I am also redundantly not exposing the routes unless we are in a test environment.

And here's the test (you may need to keep a dev/build process running for the code to be properly found by Cypress):

chapter_15/01/cypress/integration/schedule/schedule_spec.js

```
it("Allows the user to remove a favorite", () => {
  cy.request("POST", "/test/log_in_user")
  cy.request("POST", "/test/add_favorite")
  cy.visit("/")
  cy.get("#favorite-concerts-list").as("favorites")
  cy.get(".concert").first().as("concert")
  cy.get("@favorites").contains("Unfavorite").first().click()
  cy.get("@favorites").find("article").should("have.lengthOf", 0)
  cy.get("@concert").contains("Favorite")
})
```

We use cy.request twice to call each of our controller actions, then basically do the previous test in reverse: we find the first instance of an Unfavorite button, click it, then validate that the favorites section no longer has any elements in it and that the button in the existing concert is back to being labeled "Favorite."

This test passes, and you'll also note that it's way, way faster to get through the setup than the previous test that manually logs in via the actual form. It's a good idea to have at least one test that walks through the form, but other than that, you're better off doing the setup remotely.